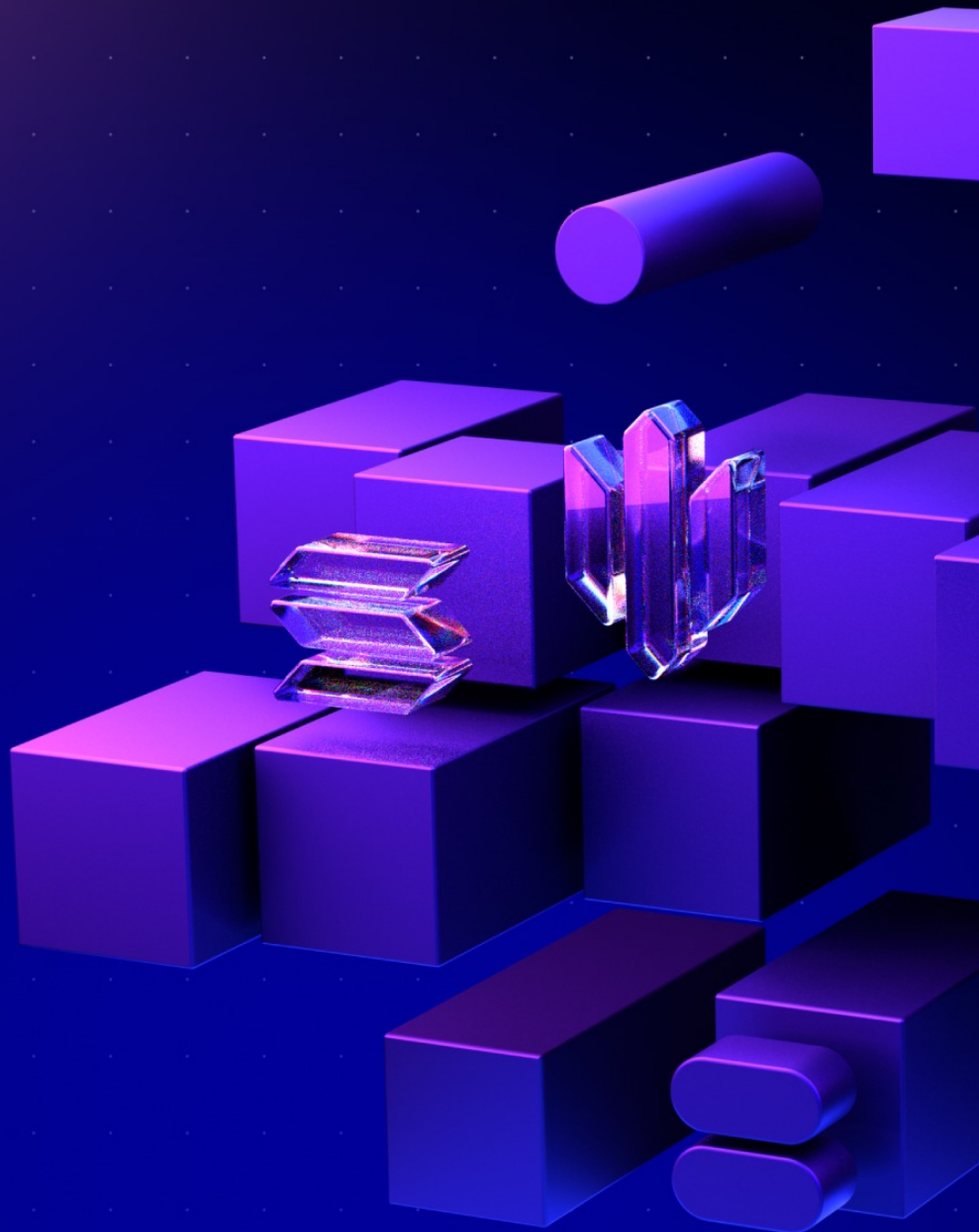


MetaDAO

The fundraising and governance platform

April 1, 2026



Contents

1. Document Revisions	4
2. Overview	5
2.1. Ackee Blockchain Security	5
2.2. Audit Methodology	6
2.3. Finding Classification	7
2.4. Review Team	9
2.5. Disclaimer	9
3. Executive Summary	10
Revision 1.0	10
Revision 1.1	12
Revision 1.2	14
Revision 2.0	15
Revision 2.1	16
4. Findings Summary	18
Report Revision 1.0	22
Revision Team	22
System Overview	22
Trust Model	23
Fuzzing	25
Findings	27
Report Revision 1.1	78
Revision Team	78
Findings	78
Report Revision 1.2	81
Revision Team	81
Report Revision 2.0	82

Revision Team	82
System Overview	82
Trust Model	83
Findings	84
Report Revision 2.1.....	90
Revision Team	90
Appendix A: How to cite	91
Appendix B: Trident Findings	92
B.1. Fuzzing.....	92
Appendix C: Trident Arena Findings	105
C.1. Discovered Findings	105

1. Document Revisions

1.0-draft	Draft Report	February 16, 2026
1.0	Final Report	February 23, 2026
1.1	Fix Review	February 24, 2026
1.2	Fix Review	February 25, 2026
2.0	Final Report	March 11, 2026
2.1	Fix Review	April 1, 2026

2. Overview

This document presents our findings in reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling [Wake](#) for Ethereum and [Trident](#) for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the [School of Solana](#) and the [Solana Auditors Bootcamp](#).

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague, Czech Republic

<https://ackee.xyz>

hello@ackee.xyz

2.2. Audit Methodology

1. Verification of technical specification

The audit scope is confirmed with the client, and auditors are onboarded to the project. Provided documentation is reviewed and compared to the audited system.

2. Manual code review

Auditors manually check the code line by line, identifying vulnerabilities and code quality issues. The main focus is on recognizing potential edge cases, project-specific risks, and common classes of Solana program vulnerabilities such as missing ownership checks, signer authorization or rent exemption assertion, signed CPI of unverified programs, cosplay of Solana accounts, bump seed canonicalization, incorrect accounts closing, casting truncation, precision errors, arithmetic overflows or underflows.

3. Local deployment and hacking

Programs are deployed locally, where targeted attempts to exploit vulnerabilities are made. The programs' resilience against various attack vectors is evaluated, with each project's attack attempts being unique to its implementation.

4. Unit and fuzz testing

Unit tests are run to verify expected system behavior. Additional unit or fuzz tests may be written using [Trident](#) framework if any coverage gaps are identified. The goal is to verify the system's stability under real-world conditions and ensure robustness against both expected and unexpected inputs.

5. Trident Arena assisted vulnerability discovery

As the last step, the scope is checked against [Trident Arena](#), an automated vulnerability analysis system for Solana program reviews, to identify potentially missed vulnerabilities. This step is executed at the end of the audit process to avoid distracting auditors from manual review.

2.3. Finding Classification

A *Severity* rating of each finding is determined as a synthesis of two sub-ratings: *Impact* and *Likelihood*. It ranges from *Informational* to *Critical*.

If we have found a scenario in which an issue is exploitable, it will be assigned an impact rating of *High*, *Medium*, or *Low*, based on the direness of the consequences it has on the system. If we haven't found a way, or the issue is only exploitable given a change in *configuration* (system settings or parameters, such as deployment scripts, compiler configurations, using multi-signature wallets for owners, etc.) or given a change in the codebase, then it will be assigned an impact rating of *Warning* or *Info*.

Low to *High* impact issues also have a *Likelihood*, which measures the probability of exploitability during runtime.

The full definitions are as follows:

Severity

		<i>Likelihood</i>			
		High	Medium	Low	N/A
<i>Impact</i>	High	Critical	High	Medium	-
	Medium	High	Medium	Low	-
	Low	Medium	Low	Low	-
	Warning	-	-	-	Warning
	Info	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue cannot be exploited given the current code and/or *configuration*, but could be a security vulnerability if these were to change slightly. If we haven't found a way to exploit the issue given the time constraints, it might be marked as a "Warning" or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations. Another example is that the issue would be security-related if code or *configuration* was to change.

Likelihood

- **High** - The issue is exploitable by virtually anyone under virtually any circumstance.
- **Medium** - Exploiting the issue currently requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

The following table lists all contributors to this report. For authors of the specific revision, see the “Revision team” section in the respective “Report revision” chapter.

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Matej Hyčko	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We've put our best effort to find all vulnerabilities in the system, however our findings shouldn't be considered as a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

3. Executive Summary

MetaDAO The fundraising and governance platform is a Solana-based decentralized autonomous organization focused on implementing futarchy, a governance model where markets, rather than direct token voting, determine which proposals should execute. It operates a platform for creating and running futarchies through decision markets and supporting on-chain primitives such as conditional assets and launch/funding mechanisms.

Revision 1.0

MetaDAO engaged Ackee Blockchain Security to perform a security review of MetaDAO The fundraising and governance platform with a total time donation of 37 engineering days in a period between December 28 and February 16, 2026, with Andrej Lukačovič as the lead auditor.

The audit was performed on the commit `6f48415`^[1] in the [MetaDAO programs](#) repository and the programs included in the audit were:

- `launchpad_v7`
- `bid_wall`
- `futarchy`
- `price_based_performance_package`
- `conditional_vault`

The audit was split into multiple phases. In the first phase, we focused on understanding the whole scope, the business model of the project, how the frontend part operates and what actually is the intentional behavior of the project. Within this phase we were communicating with the client to clarify our assumptions. The first phase was also dedicated to implementing simple integration tests, using the [Trident](#) framework, which helped us to understand the communication with the onchain programs, what are all the

instruction parameters, accounts and how to interact with them.

The second phase was dedicated to the in-depth analysis of the code and writing more sophisticated fuzz tests. For fuzzing, we utilized the [Trident](#) framework. During the manual review we paid special attention to the following:

- ensuring the math operations are correctly implemented;
- ensuring all of the accounts are correctly constrained and no misuse is possible;
- ensuring no one can benefit from any truncating or overflowing operations;
- ensuring the communication between all programs in scope is correctly implemented and no misuse is possible; and
- ensuring the 3rd party programs are used as expected;

The fuzzing yielded the [C1](#) issue, where separate deposits and withdrawals model was implemented within the fuzz test to ensure the correctness of the implementation.

More details about the fuzzing can be found in Fuzzing section of [Report Revision 1.0](#). The list of implemented execution flows and invariants is available in [Appendix B](#).

The third phase of the audit focused on consolidating our findings, developing proof-of-concept tests where necessary to demonstrate identified vulnerabilities, and compiling the audit report.

As the final phase of the audit, we employed [Trident Arena](#), a multi-agent AI system, which provided additional analysis and verification of the identified issues. See the [Appendix C](#) for the complete list of findings identified by [Trident Arena](#).

Our review resulted in 19 findings, ranging from Info to Critical severity.

The most severe one [C1](#) is a critical severity issue caused by truncation when casting a `u128` to a `u64`. Truncated values can be used in denominators when computing withdrawable amounts, enabling an attacker to withdraw more assets than they provided as liquidity.

The [H1](#) is a high severity issue stemming from `init_if_needed` usage with two liquidity position authorities. An attacker could re-initialize a user's position authority with a minimal deposit, potentially preventing withdrawals until the user resets authority.

The [H2](#) is a high severity issue where oracle configuration can be changed during an active unlock cycle. Changing configuration between `start_unlock` and `complete_unlock` could skew TWAP and unlock performance package tranches under inconsistent oracle data.

Ackee Blockchain Security recommends MetaDAO:

- read and review the complete audit report; and
- address all identified issues.

See [Report Revision 1.0](#) for the system overview and trust model.

Revision 1.1

MetaDAO engaged Ackee Blockchain Security to perform a fix review of the findings from the previous revision.

The review was performed between February 23 and February 24, 2026.

MetaDAO provided multiple commits that addressed the reported findings.

The issue [C1](#) was fixed in commit `ee7dd1d`^[2] by removing the downcast and using `u128` arithmetic instead.

The issue [H1](#) was fixed in commit [8748a38](#)^[3] by updating the position authority only when the initial liquidity is provided. Subsequent calls to `provide_liquidity` do not update the position authority; they only add liquidity to the position.

The issue [H2](#) was fixed in commit [3030762](#)^[4] by adding a check to ensure that the oracle configuration change can neither be proposed nor executed during an unlock period.

The issue [L1](#) was partially fixed in commit [d7fc89a](#)^[5] by returning the bid wall account rent to the authority. However, the `bid_wall_quote_token_account` token account is still closed to the `payer`.

The issue [L2](#) was fixed in commit [896a411](#)^[6] by updating the TWAP calculation to use the prior observation for the entire interval instead of the new observation.

The issue [W3](#) was fixed in commit [3865e80](#)^[7] by checking whether the question was resolved before processing operations.

The issue [W4](#) was fixed in commit [66e4a02](#)^[8] by allowing the DAO to be updated only when it was in the Spot state.

The issue [I1](#) was fixed in commit [68aecdd1](#)^[9] by removing the unnecessary linear search inside a for loop.

The issue [I2](#) was fixed in commit [ebb8c7f](#)^[10] by preventing the creation of a DAO with identical base and quote mints.

The issue [I3](#) was fixed in commit [ebb8c7f](#)^[11] by changing the error message to "Squads proposal must be in Active status" to match the validation.

The issue [I5](#) was fixed in commit [cb2c17b](#)^[12] by adding a check to ensure that `recipient` and `performance_package_authority` are distinct keys during initialization, authority change, and change request execution.

Additionally, one new issue was reported: [W7](#).

From the reported 19 findings:

- 8 issues were fixed: [C1](#), [H1](#), [L2](#), [W3](#), [W4](#), [I1](#), [I2](#), and [I3](#).
- 2 issues were acknowledged: [W1](#), [W2](#)
- 1 issue was partially fixed: [L1](#).
- 0 issues remained unresolved.

No new findings were reported.

See [Report Revision 1.1](#) for the complete fix review.

Revision 1.2

MetaDAO engaged Ackee Blockchain Security to perform a fix review of the findings from the previous revision.

The review was performed between February 25 and February 25, 2026.

MetaDAO provided a commit `5fec0a9`^[13] with the fix.

The issue [W7](#) was fixed by relaxing the check, which now verifies that at least one oracle update happened after the TWAP update delay (`start_delay_seconds`) ended.

From the reported 1 findings:

- 1 issue was fixed: [W7](#).
- 0 issues remained unresolved.

No new findings were reported.

See [Report Revision 1.2](#) for the complete fix review.

Revision 2.0

MetaDAO engaged Ackee Blockchain Security to perform a security review of MetaDAO The fundraising and governance platform with a total time donation of 5 engineering days in a period between March 3 and March 11, 2026, with Andrej Lukačovič as the lead auditor.

The audit was performed on the commit `4a25361`^[14] in the [MetaDAO programs](#) repository and the programs included in the audit were:

- `mint_governor`
- `performance_package_v2`

The audit was split into multiple phases. In the first phase, we focused on understanding the intended behavior, trust assumptions, and privileged flows of the Mint Governor and Performance Package V2 programs. This phase established how mint delegation, package configuration, and token release mechanisms are expected to operate, along with the main trust dependencies introduced by each design.

The second phase was dedicated to the in-depth analysis of the code and targeted testing of the most security-sensitive execution paths. During the manual review we paid special attention to the following:

- ensuring privileged flows are properly restricted;
- ensuring all accounts are correctly constrained and cannot be misused;
- ensuring minting, authority management, package updates, and token release flows cannot be abused by unauthorized actors; and
- ensuring state transitions cannot bypass intended security guarantees.

For Mint Governor, we focused on authority delegation, mint authorization records, and privileged minting flows, especially the risk of unauthorized token issuance. For Performance Package V2, we focused on package

initialization, permissions, update semantics, and token release logic, especially the risk of unfair fund deprivation or unauthorized changes.

The third phase of the audit focused on consolidating our findings, validating issues through proof-of-concept scenarios where necessary, and compiling the audit report.

In the final phase, we used [Trident Arena](#), a multi-agent AI system, as an additional review layer to further analyze the codebase and cross-check the identified issues. The complete list of findings reported by [Trident Arena](#) is provided in [Appendix C](#).

Our review resulted in 3 findings, ranging from Info to Warning severity.

Ackee Blockchain Security recommends MetaDAO:

- read and review the complete audit report; and
- address all identified issues.

See [Report Revision 2.0](#) for the system overview and trust model.

Revision 2.1

MetaDAO engaged Ackee Blockchain Security to perform a fix review of the findings from the previous revision.

The review was performed between March 30 and March 31, 2026.

From the reported 3 findings:

- 3 issues were acknowledged: [W9](#), [W8](#), and [I7](#).
- 0 issues remained unresolved.

No new findings were reported.

See [Report Revision 2.1](#) for the complete fix review.

- [1] full commit hash: 6f4841515b04d906018d5f71a0b6ce373335665b
- [2] full commit hash: ee7dd1dd5cc0f15027a799f1753c6e9b6c96a98a
- [3] full commit hash: 8748a384f9ce92ef3fafcbb9f0aac0a64863c5e8
- [4] full commit hash: 30307620cdb5fb3ef23a5a0a0bcf1043c2b9c460
- [5] full commit hash: d7fc89a7dcfb66c05106d0bcb07959c66c8e78d6
- [6] full commit hash: 896a4117662d64b223afb912f4c35535233a6324
- [7] full commit hash: 3865e80491c3a2005e3515b4d3bd58551e6b91a3
- [8] full commit hash: 66e4a0226c3d42437200e5a2365337d6bc7c1999
- [9] full commit hash: 68aec19e58c8fbcc2f3861f6291b74c278678b2
- [10] full commit hash: ebb8c7fc85c85310ffc5d948884220a549e6c8ae
- [11] full commit hash: ebb8c7fc85c85310ffc5d948884220a549e6c8ae
- [12] full commit hash: cb2c17bdc952629625a2a82b299ea29a17b2b81e
- [13] full commit hash: 5fec0a93a498332be443bff09ad9868a5008e6c4
- [14] full commit hash: 4a25361dac367a36ff726f88139dd38d75b20e99

4. Findings Summary

The following section summarizes findings we identified during our review. Unless overridden for purposes of readability, each finding contains:

- *Description*
- *Exploit scenario* (if severity is low or higher)
- *Recommendation*
- *Fix* (if applicable).

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
1	2	2	2	9	7	23

Table 2. Findings Count by Severity

Findings in detail:

Finding title	Severity	Reported	Status
C1 : u128→u64 downcast during withdraw_liquidity corrupts share math and can cause division-by-zero	Critical	1.0	Fixed
H1 : LP Position Seeds/Authority Mismatch Bricks Withdrawals	High	1.0	Fixed
H2 : Oracle configuration can be changed during an unlock period, enabling TWAP manipulation	High	1.0	Fixed
M1 : Stale change requests can be executed after recipient or authority changes	Medium	1.0	Acknowledged

Finding title	Severity	Reported	Status
M2 : Oracle configuration changes allow using an untrusted oracle account for TWAP-based unlocking	Medium	1.0	Acknowledged
L1 : Rent from closed bid wall accounts can be claimed by unauthorized users	Low	1.0	Partially fixed
L2 : TWAP accumulator uses new_observation for the prior interval, enabling retroactive per-minute weighting under 60s update gating	Low	1.0	Fixed
W1 : Compile-time gated admin check allows unauthorized fee collection when the production feature is disabled	Warning	1.0	Acknowledged
W2 : Multiple instructions allow zero-amount operations causing unnecessary state updates and event emissions	Warning	1.0	Acknowledged
W3 : Missing resolved check in split_tokens and merge_tokens allows operations on resolved questions	Warning	1.0	Fixed

Finding title	Severity	Reported	Status
W4: DAO pass thresholds are not snapshotted per-proposal, allowing governance to change proposal finalization criteria mid-flight	Warning	1.0	Fixed
W5: Hardcoded fee recipient address prevents rotating the Bid Wall fee destination without redeploying	Warning	1.0	Acknowledged
W6: Single-step authority transfer	Warning	1.0	Acknowledged
I1: $O(N^2)$ linear search in <code>redeem_tokens</code> causes unnecessary compute usage	Info	1.0	Fixed
I2: Missing validation in <code>initialize_dao</code> allows identical base and quote mints causing accounting inconsistencies	Info	1.0	Fixed
I3: Incorrect error message in <code>initialize_proposal</code> contradicts Active status validation	Info	1.0	Fixed
I4: Performance packages cannot be closed after all tranches are unlocked, preventing rent recovery	Info	1.0	Acknowledged
I5: Recipient and authority can be set to the same key	Info	1.0	Fixed

Finding title	Severity	Reported	Status
I6 : Anyone can complete unlock, allowing third parties to choose the TWAP evaluation time	Info	1.0	Acknowledged
W7 : Proposal finalization depends on a post-deadline TWAP update	Warning	1.1	Fixed
W8 : Rent destination is not strictly enforced to be the initial rent payer	Warning	2.0	Acknowledged
W9 : Single step authority transfer process is considered unsafe	Warning	2.0	Acknowledged
I7 : Safety checks are disabled when the production feature is not enabled	Info	2.0	Acknowledged

Table 3. Table of Findings

Report Revision 1.0

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Matej Hyčko	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

MetaDAO The fundraising and governance platform is a futarchy-based governance platform on Solana that enables decision-making through prediction markets.

Conditional Vault manages conditional tokens representing outcomes of binary questions (e.g., "Will proposal X pass?"). Users can split underlying tokens into conditional PASS/FAIL tokens, merge them back, or redeem them after resolution based on the outcome's payout ratio.

Futarchy is the main governance program that creates DAOs with embedded AMMs for trading conditional tokens. Proposals go through a lifecycle: Draft → Pending (after staking) → Passed/Failed (based on TWAP price comparison). If the PASS market's time-weighted average price exceeds the FAIL market's price by a configurable threshold, the proposal passes and executes via Squads multisig.

Launchpad V7 facilitates the creation of new futarchic DAOs through token launches. It manages a funding period where users commit USDC, and upon reaching the minimum raise, it mints governance tokens, creates the DAO, and sets up liquidity pools on Meteora DAMM V2.

Bid Wall acts as a buyback mechanism that sits at NAV (Net Asset Value),

allowing token holders to sell their tokens back to the protocol at a fair price. It burns purchased tokens, reducing supply and providing downside protection for holders.

Price-Based Performance Package is a token vesting program that unlocks tokens to recipients when price thresholds are met. Tokens are split into up to 10 tranches, each unlocking at different price levels determined by TWAP oracles, aligning team incentives with token performance.

The system creates a complete governance ecosystem where market prices—rather than simple voting—determine which proposals benefit the organization, with supporting infrastructure for token launches, liquidity, and performance-based compensation.

Trust Model

Futarchy

The Futarchy program implements DAO governance through prediction markets, where proposal outcomes are determined by market prices.

In the Futarchy program, users must trust that:

- the **Squads Multisig Vault** will not manipulate DAO parameters (pass thresholds, TWAP settings, liquidity requirements) to force or block proposals;
- the **Team Address** will not unfairly sponsor proposals to give them advantageous pass thresholds;
- the **Hardcoded Admin** (`6awyHM...`, `tSTp6B...`) will collect protocol fees appropriately; and
- the TWAP oracle mechanism is not manipulable during proposal voting periods.

Bid Wall

The Bid Wall program provides a price floor mechanism where users can sell tokens at a guaranteed minimum price.

In the Bid Wall program, users must trust that:

- the **Authority** will not cancel the bid wall prematurely, removing the price floor before users can sell;
- the fee and price parameters set at initialization are fair, as they are immutable after creation; and
- the **Fee Recipient** will receive fees appropriately (fee collection is permissionless).

Launchpad v7

The Launchpad program manages token launches with funding approval mechanisms and post-launch token distribution.

In the Launchpad v7 program, users must trust that:

- the **Launch Authority** will not reject legitimate funders to exclude them from token distribution;
- the **Launch Authority** will approve funding contributions fairly within the 2-day window after launch closes;
- the **Launch Authority** will complete the launch appropriately (or users must wait 2 days for permissionless completion);
- the token allocation parameters and additional token recipient set at initialization are accurate; and
- the **Additional Tokens Recipient** is a legitimate party entitled to extra token allocations.

Conditional Vault

The Conditional Vault program manages conditional tokens for prediction markets, where token redemption depends on question resolution.

In the Conditional Vault program, users must trust that:

- the **Oracle** will resolve questions honestly and in a timely manner;
- the **Oracle** key is set correctly at question creation, as it cannot be changed afterward; and
- the **Prop3t Deployer** (in production) will add accurate metadata to conditional tokens.

Price-Based Performance Package

The Price-Based Performance Package program implements token vesting with price-based unlock conditions and a mutual consent model for changes.

In the Price-Based Performance Package program, users must trust that:

- the **Performance Package Authority** will execute legitimate recipient-proposed changes;
- the **Recipient** will execute legitimate authority-proposed changes;
- the oracle configuration for price-based unlocks is accurate and not manipulable; and
- the mutual consent model (changes require both parties) functions correctly.

Fuzzing

Fuzz testing is an automated testing technique that discovers bugs by providing invalid, unexpected, or random data as inputs to a Solana program.

Fuzz testing helps identify edge cases and vulnerabilities that might be missed during manual review.

Manually Guided Fuzz Testing was utilized to test the correctness and robustness of the system. This approach involves manually guiding the fuzzer by specifying which Solana instruction sequences should be executed and what accounts should be used. The essence of this approach is to ensure that the fuzzer is guided by the intended behavior of the system and not just randomly exploring the codebase. This helps the auditors to focus on the most critical parts of the codebase and not waste time on exploring the codebase randomly.

For this audit revision, pseudo fork testing was utilized to create a testing environment that closely mimics the Solana Mainnet. This technique involves dumping real Solana accounts and programs from the Mainnet (such as on-chain accounts storing asset prices) and loading them into the TridentSVM execution environment. This approach ensures that the fuzz tests interact with realistic account data and program dependencies. The pseudo designation indicates that while we replicate Mainnet state, Trident cannot yet automatically fetch all required accounts from the desired cluster, requiring manual cloning of necessary Solana accounts and programs.

Additionally, property-based fuzz testing was employed to verify the correctness and expected behavior of Solana program instructions. This approach involves writing invariant checks—mathematical properties that must always hold true—which are automatically validated after each instruction executes successfully. These invariants allow us to compare the actual program behavior against expected behavior, catching subtle logic errors and account state inconsistencies. The fuzz testing campaign simulated a comprehensive spectrum of Solana system instructions while enforcing strong assertions about the program's behavior.

The complete list of implemented execution flows and invariants is available

in [Appendix B](#).

Findings

The following section presents the list of findings discovered in this revision.
For the complete list of all findings, [Go back to Findings Summary](#)

C1: u128→u64 downcast during `withdraw_liquidity` corrupts share math and can cause division-by-zero

Critical severity issue

Impact:	High	Likelihood:	High
Target:	programs/futarchy/src/instructions/withdraw_liquidity.rs, programs/futarchy/src/state/futarchy_amm.rs	Type:	Logic error

Description

The `withdraw_liquidity` instruction casts 128-bit liquidity values to 64 bits before calculating withdrawable amounts. Both `liquidity_to_withdraw` and `dao.amm.total_liquidity` are stored as `u128`, but the code converts them to `u64` and passes them to `Pool::get_base_and_quote_withdrawable`. This truncation corrupts the LP ratio when values exceed `u64::MAX` and leads to two critical issues:

- Inflated withdrawal amounts when `total_liquidity` is truncated in the denominator, allowing users to withdraw more tokens than they should; and
- Division-by-zero panic when the truncated `lp_total_supply` becomes zero while the actual `total_liquidity` is non-zero, creating a Denial of Service (DoS) condition.

The issue starts in the `withdraw_liquidity` instruction, where the `total_liquidity` is cast to `u64` and passed to the `Pool::get_base_and_quote_withdrawable` helper.

Listing 1. Excerpt from withdraw_liquidity

```
94 let (base_to_withdraw, quote_to_withdraw) = {
95     let PoolState::Spot { ref spot } = dao.amm.state else {
96         // TODO: check that pool is already in right state
97         unreachable!();
98     };
99     spot.get_base_and_quote_withdrawable(
100         liquidity_to_withdraw as u64,
101         total_liquidity as u64,
102     )
103 };
```

The `Pool::get_base_and_quote_withdrawable` helper is defined with `u64` parameters and calls the `Pool::get_base_withdrawable` and `Pool::get_quote_withdrawable` helpers. Both of these helpers perform division with the truncated `total_liquidity` value.

Listing 2. Excerpt from futarchy_amm

```
584 pub fn get_base_and_quote_withdrawable(
585     &self,
586     lp_tokens: u64,
587     lp_total_supply: u64,
588 ) -> (u64, u64) {
589     (
590         self.get_base_withdrawable(lp_tokens, lp_total_supply),
591         self.get_quote_withdrawable(lp_tokens, lp_total_supply),
592     )
593 }
594
595 /// Get the number of base tokens withdrawable from a position
596 pub fn get_base_withdrawable(&self, lp_tokens: u64, lp_total_supply: u64) ->
597     u64 {
598     // must fit back into u64 since `lp_tokens` <= `lp_total_supply`
599     ((lp_tokens as u128 * self.base_reserves as u128) / lp_total_supply as
600         u128) as u64
601 }
602
603 /// Get the number of quote tokens withdrawable from a position
604 pub fn get_quote_withdrawable(&self, lp_tokens: u64, lp_total_supply: u64)
605     -> u64 {
606     ((lp_tokens as u128 * self.quote_reserves as u128) / lp_total_supply as
```

```
u128) as u64
604 }
```

The initial liquidity is set to `quote_amount * 1_000_000_000`, which grows quickly and can exceed 64 bits.

Listing 3. Excerpt from `provide_liquidity`

```
123 let initial_liquidity = quote_amount as u128 * 1_000_000_000;
```

Exploit scenario

Case A — Division-by-zero DoS

Alice, a user, creates a new pool and deposits an initial liquidity.

1. Alice coordinates deposits so that `total_liquidity % 2^64 == 0`;
2. Bob, another user, calls `withdraw_liquidity`;
3. The program casts `total_liquidity` to `u64`, which becomes 0, and `get_*_withdrawable` divides by 0, causing a panic; and
4. Until the pool state changes (e.g., a new deposit alters `total_liquidity`), all withdrawal attempts fail.

Case B — Inflated withdrawal amounts

Alice, a user, creates a new pool and deposits an initial liquidity so that `total_liquidity` exceeds 64 bits.

1. Bob, a malicious actor, observes the `total_liquidity` value and calculates that after truncation to `u64`, the denominator will be very small (potentially as low as 1);
2. Bob deposits liquidity such that when `total_liquidity` is cast to `u64`, the truncated value is minimal;

3. Bob calls `withdraw_liquidity` with his LP position; and
4. The program casts `total_liquidity` to `u64`, which truncates it to a very small value in the denominator, resulting in `base_to_withdraw` and `quote_to_withdraw` being much larger than the correct proportional amounts, allowing Bob to withdraw more tokens than he should and extract value from the pool.

Recommendation

Remove the `u128` to `u64` downcast in `withdraw_liquidity`. Update `Pool::get_base_and_quote_withdrawable` and related helper functions to accept `u128` parameters and perform calculations using `u128` arithmetic. Add a non-zero check for `lp_total_supply` before division operations to prevent division-by-zero panics.

Fix 1.1

The issue was fixed by removing the downcast and using `u128` arithmetic instead.

[Go back to Findings Summary](#)

H1: LP Position Seeds/Authority Mismatch Bricks Withdrawals

High severity issue

Impact:	Medium	Likelihood:	High
Target:	programs/futarchy/src/instructions/provide_liquidity.rs, programs/futarchy/src/instructions/withdraw_liquidity.rs	Type:	Data validation

Description

When providing liquidity, the position address (PDA) is derived from `params.position_authority`, but the stored authority field is overwritten with the signer's key (`liquidity_provider.key()`) without validation. This allows an attacker to deposit a small amount into an existing position and overwrite the authority, making withdrawals impossible until the original owner re-initializes the position.

The issue occurs in the `ProvideLiquidity` instruction, which lacks validation to ensure `params.position_authority` matches `liquidity_provider.key()`. Additionally, there is no `has_one` constraint on `amm_position` to prevent this mismatch. In contrast, `WithdrawLiquidity` correctly derives the PDA with the signer's key and enforces `has_one = position_authority`.

The PDA is derived using `params.position_authority` as a seed:

Listing 4. Excerpt from provide_liquidity

```
50 #[account(  
51     init_if_needed,  
52     payer = payer,  
53     seeds = [b"amm_position", dao.key().as_ref(),  
               params.position_authority.key().as_ref()],
```

```

54     bump,
55     space = 8 + AmmPosition::INIT_SPACE,
56 )]
57 pub amm_position: Account<'info, AmmPosition>,

```

However, the `AmmPosition.position_authority` field is overwritten with the signer's key (`liquidity_provider.key()`) without checking if it matches the seed used for PDA derivation. Combined with the `init_if_needed` constraint, this allows an attacker to deposit a minimum amount and overwrite the authority to their own key:

Listing 5. Excerpt from `provide_liquidity`

```

131 amm_position.set_inner(AmmPosition {
132     dao: dao.key(),
133     position_authority: liquidity_provider.key(),
134     liquidity: amm_position.liquidity + liquidity_to_mint,
135 });

```

The impact is marked as Medium because funds are not locked forever; the victim can call `provide_liquidity` again to set themselves as the authority and withdraw the funds.

Exploit scenario

Alice, a victim, already has a position at `P = PDA("amm_position", dao, Alice)`.

1. Bob, a malicious actor, calls `provide_liquidity` with `liquidity_provider = Bob` (signer) and `params.position_authority = Alice` (a Pubkey) and deposits a small amount of funds;
2. The program derives `amm_position = P` from seeds and overwrites `AmmPosition.position_authority` with Bob, while crediting more liquidity to P;
3. Later withdrawals at P fail for everyone: Alice fails `has_one = position_authority` (stored field is Bob), and Bob cannot satisfy the PDA

seeds (which encode Alice), making the position unwithdrawable; and

4. Alice needs to call `provide_liquidity` again to set themselves as the authority and withdraw the funds.

Recommendation

Remove two parameters from the `provide_liquidity` instruction. The signer should be used as a seed for the PDA and simultaneously stored as the position authority.

Fix 1.1

The issue was fixed by updating the position authority only when the initial liquidity is provided, the subsequent calls to `provide_liquidity` do not update the position authority, only the liquidity is added to the position.

[Go back to Findings Summary](#)

H2: Oracle configuration can be changed during an unlock period, enabling TWAP manipulation

High severity issue

Impact:	High	Likelihood:	Medium
Target:	programs/price_based_performance_package/src/instructions/complete_unlock.rs	Type:	Logic error

Description

The unlock flow uses `oracle_config` twice: `start_unlock` reads the starting aggregator value and stores it in `PerformancePackageState::Unlocking`.

`complete_unlock` later reads the current aggregator value from the oracle specified by the (current) `performance_package.oracle_config` and computes the TWAP using the stored start values:

Listing 6. Excerpt from complete_unlock

```
83 // Get the start values from the Unlocking state
84 let (start_aggregator, start_timestamp) = match &performance_package.state {
85     PerformancePackageState::Unlocking {
86         start_aggregator,
87         start_timestamp,
88     } => (*start_aggregator, *start_timestamp),
89     _ => unreachable!(),
90 };
91
92 // Read the current aggregator value from the oracle account
93 let oracle_data = oracle_account.try_borrow_data()?;
94 let offset = performance_package.oracle_config.byte_offset as usize;
95
96 // Ensure we have enough data to read 16 bytes (u128)
97 require_gte!(
98     oracle_data.len(),
99     offset + 24,
100     PriceBasedPerformancePackageError::InvalidOracleData
```

```

101 );
102
103 // Read the current aggregator value
104 let current_aggregator =
105     u128::from_le_bytes(oracle_data[offset..offset +
106     16].try_into().unwrap());
107
108 let last_updated_timestamp = i64::from_le_bytes(
109     oracle_data[offset + 16..offset + 16 + 8]
110     .try_into()
111     .unwrap(),
112 );
113
114 require_gte!(
115     clock.unix_timestamp,
116     last_updated_timestamp,
117     PriceBasedPerformancePackageError::InvalidOracleData
118 );
119
120 let time_passed = last_updated_timestamp - start_timestamp;
121
122 require_gte!(
123     time_passed,
124     performance_package.twap_length_seconds as i64,
125     PriceBasedPerformancePackageError::TwapPeriodNotElapsed
126 );
127
128 // Calculate TWAP: (current_aggregator - start_aggregator) / time_passed
129 let aggregator_change = current_aggregator.saturating_sub(start_aggregator);
130 let twap_price = aggregator_change / time_passed as u128;

```

However, `oracle_config` can be updated via the change-request mechanism without any state restriction, including while the performance package is in the `Unlocking` state. `execute_change` applies the new oracle configuration directly.

As a result, `start_unlock` can capture `start_aggregator` and `start_timestamp` from one oracle configuration, while `complete_unlock` reads `current_aggregator` and `last_updated_timestamp` from a different oracle configuration. This can break the TWAP calculation.

Exploit scenario

Alice, a token recipient, wants to unlock tokens without the intended TWAP conditions. Bob, the performance package authority, can execute changes proposed by Alice.

1. Alice calls `start_unlock` while `oracle_config` points to oracle O1, which has an old `last_updated_timestamp`, and the program stores `start_aggregator` and `start_timestamp` from O1 in `PerformancePackageState::Unlocking`;
2. Alice proposes a change that updates `oracle_config` to oracle O2; and
3. Bob executes the change via `execute_change`, updating `performance_package.oracle_config` while the package is still in the `Unlocking` state.

When Alice (or Bob) calls `complete_unlock`, the program reads `current_aggregator` and `last_updated_timestamp` from O2, but it computes the TWAP using the start values from O1. This mismatch can produce an arbitrarily high TWAP value and allow unlocking tranches that would not unlock under a single consistent oracle configuration.

Recommendation

Prevent oracle configuration changes during an unlock period.

Fix 1.1

The issue was fixed by adding a check to ensure that the oracle configuration change can neither be proposed nor executed during an unlock period.

[Go back to Findings Summary](#)

M1: Stale change requests can be executed after recipient or authority changes

Medium severity issue

Impact:	High	Likelihood:	Low
Target:	programs/price_based_perfor mance_package/src/instructi ons/execute_change.rs	Type:	Logic error

Description

The change-request flow uses `propose_change` to create a `ChangeRequest` PDA that is seeded by the `proposer` key. However, the `ChangeRequest` account does not store the proposer key or snapshot the recipient/authority keys at proposal time. It only stores `proposer_type`.

`execute_change` validates only that the executor matches the **current** recipient or authority, depending on `proposer_type`, and it does not enforce any PDA seed constraint tying the `change_request` to the current recipient or authority:

Listing 7. Excerpt from `execute_change`

```
7  #[derive(Accounts)]
8  pub struct ExecuteChange<'info> {
9      #[account(
10         mut,
11         has_one = performance_package @
12         PriceBasedPerformancePackageError::InvalidChangeRequest,
13         close = executor
14     )]
15     pub change_request: Account<'info, ChangeRequest>,
16
17     #[account(mut)]
18     pub performance_package: Account<'info, PerformancePackage>,
19
20     /// The party executing the change (must be opposite of proposer)
```

```

20     #[account(mut)]
21     pub executor: Signer<'info>,
22 }
23
24 impl<'info> ExecuteChange<'info> {
25     pub fn validate(&self) -> Result<()> {
26         if self.change_request.proposer_type == ProposerType::Recipient {
27             // If recipient proposed, locker authority must execute
28             require_keys_eq!(
29                 self.executor.key(),
30                 self.performance_package.performance_package_authority,
31
32                 PriceBasedPerformancePackageError::UnauthorizedLockerAuthority
33             );
34         } else if self.change_request.proposer_type ==
35             ProposerType::Authority {
36             // If authority proposed, recipient must execute
37             require_keys_eq!(
38                 self.executor.key(),
39                 self.performance_package.recipient,
40                 PriceBasedPerformancePackageError::UnauthorizedChangeRequest
41             );
42         } else {
43             // Proposer was neither valid party - should not happen due to
44             // proposal constraints
45             return
46                 Err(PriceBasedPerformancePackageError::UnauthorizedChangeRequest.into());
47         }
48     }
49     Ok(())
50 }

```

As a result, a stale **ChangeRequest** that was proposed by a prior recipient (or prior authority) can remain executable even after the recipient (or authority) has changed.

Exploit scenario

Alice, a token recipient, wants a previously proposed change to be executed after she is no longer the recipient. Bob, the performance package authority, can execute changes proposed by a token recipient.

1. Alice proposes a `ChangeType::Recipient` change request (e.g., setting `new_recipient = Alice`) while she is the current recipient;
2. Later, the performance package recipient is changed to a different key; and
3. Bob executes Alice's previously proposed change request via `execute_change`.

Because `execute_change` does not validate that the change request was proposed by the **current** recipient, Bob can execute the stale request and set the recipient according to Alice's prior proposal without approval from the new recipient.

Recommendation

Bind change requests to the current recipient/authority at proposal time and reject stale requests.

Acknowledgment 1.1

The client stated that the current behavior is intended and will not be changed and provided the following comment:

The program originally stored the full proposer key in `ChangeRequest`, which is exactly what the auditor recommends. We intentionally replaced it with `proposer_type` in commit 7e22019 (Sep 23, 2025) because binding change requests to a specific key caused legitimate in-flight change requests to become permanently un-executable after role rotations. The current behavior is a deliberate design choice, not an oversight.

In practice, the performance package authority is the DAO's

squads_multisig_vault, meaning initiating a stale change request requires a full governance proposal to pass. Even then, the change request flow requires the counterparty (the grantee) to accept. The exploit scenario described by the auditor – where the authority unilaterally executes a stale request – cannot happen without both DAO-level consensus and counterparty approval.

— MetaDAO team

[Go back to Findings Summary](#)

M2: Oracle configuration changes allow using an untrusted oracle account for TWAP-based unlocking

Medium severity issue

Impact:	High	Likelihood:	Low
Target:	programs/price_based_performance_package/src/instructions/execute_change.rs	Type:	Data validation

Description

The `PerformancePackage` stores an `OracleConfig` (`oracle_account` and `byte_offset`) and uses it as the data source for TWAP-based unlocking. However, oracle configuration updates do not validate that `oracle_account` is a valid oracle account owned by an expected oracle program.

In `execute_change`, the new configuration is assigned directly:

Listing 8. Excerpt from `execute_change`

```
52 // Apply the change based on type
53 match &change_request.change_type {
54     ChangeType::Oracle { new_oracle_config } => {
55         performance_package.oracle_config = *new_oracle_config;
56     }
57     ChangeType::Recipient { new_recipient } => {
58         performance_package.recipient = *new_recipient;
59     }
60 }
```

As a result, a configuration change can point `oracle_account` to an arbitrary account, and subsequent calls to `start_unlock/complete_unlock` use raw bytes from that account as the TWAP input.

Exploit scenario

Alice, a token recipient, wants to unlock tranches without the intended oracle-backed TWAP conditions. Bob, the performance package authority, can execute changes proposed by the token recipient.

1. Alice deploys (or controls) a program that maintains a data account `x` with attacker-controlled bytes matching the expected layout (aggregator `u128` and timestamp `i64`) at some offset;
2. Alice proposes a change request that updates `oracle_config.oracle_account` to `x` (and sets `byte_offset` accordingly);
3. Bob executes the change via `execute_change`, updating the stored `oracle_config` to point to `x`;
4. Alice calls `start_unlock`, supplying `oracle_account = x`, so the program records `start_aggregator` and `start_timestamp` from attacker-controlled bytes; and
5. After enough time passes, Alice updates the bytes in `x` to yield an arbitrarily large TWAP and calls `complete_unlock`, causing the program to unlock tranches based on manipulated oracle data.

Recommendation

Validate oracle configuration changes by constraining `oracle_account` to an expected oracle program and validating the account layout/offset. If the program cannot validate the oracle account without passing it as an account to the change instruction, require the `oracle_account` to be provided to `propose_change/execute_change` and validate it there.

Acknowledgment 1.1

The client stated that the current behavior is intended and will not be changed and provided the following comment:

The oracle-agnostic design is intentional. initialize_performance_package also accepts oracle_config with zero validation – no owner check, no program ID constraint, no layout validation. The OracleConfig struct is simply a Pubkey and a byte_offset, and the program intentionally reads raw bytes at an arbitrary offset from any account. Adding validation only at change time would be inconsistent with initialization and provide no meaningful security improvement.

The security boundary here is the two-party consent mechanism in the change request flow: the proposer and executor must be opposite parties (if the authority proposes, the recipient must execute, and vice versa). The auditor's exploit scenario requires the counterparty to blindly execute a change request pointing to a fake oracle account. If the counterparty is performing due diligence – which the two-party design presupposes – this is not exploitable. In practice, the authority is the DAO's squads_multisig_vault, so changing the oracle requires a full governance proposal to pass.

— MetaDAO team

[Go back to Findings Summary](#)

L1: Rent from closed bid wall accounts can be claimed by unauthorized users

Low severity issue

Impact:	Low	Likelihood:	Medium
Target:	programs/bid_wall/src/instructions/cancel_bid_wall.rs, programs/bid_wall/src/instructions/close_bid_wall.rs	Type:	Data validation

Description

The `close_bid_wall` instruction allows any user to close an expired or depleted bid wall and receive the rent refund from both the bid wall account and its associated quote token account. The `payer` who receives the rent is not validated to be the original payer who funded the account creation, nor is it required to be the authority.

During initialization, the `payer` funds both accounts:

Listing 9. Excerpt from initialize_bid_wall

```
25 #[account(  
26     init,  
27     payer = payer,  
28     space = 8 + BidWall::INIT_SPACE,  
29     seeds = [b"bid_wall", base_mint.key().as_ref(), creator.key().as_ref(),  
30               args.nonce.to_le_bytes().as_ref()],  
31     bump  
32 )]  
32 pub bid_wall: Account<'info, BidWall>,
```

Listing 10. Excerpt from initialize_bid_wall

```
46 #[account(init_if_needed, payer = payer, associated_token::mint = quote_mint,  
            associated_token::authority = bid_wall)]
```

```
47 pub bid_wall_quote_token_account: Account<'info, TokenAccount>,
```

However, during closure, the rent is sent to whoever provides the `payer` account in the transaction:

Listing 11. Excerpt from `close_bid_wall`

```
14 #[account(  
15     mut,  
16     close=payer,  
17     has_one = authority  
18 )]  
19 pub bid_wall: Account<'info, BidWall>,  
20  
21 #[account(mut)]  
22 pub payer: Signer<'info>,
```

Listing 12. Excerpt from `close_bid_wall`

```
118 // Close the bid wall quote ATA  
119 token::close_account(CpiContext::new_with_signer(  
120     ctx.accounts.token_program.to_account_info(),  
121     token::CloseAccount {  
122         account:  
123             ctx.accounts.bid_wall_quote_token_account.to_account_info(),  
124             destination: ctx.accounts.payer.to_account_info(),  
125             authority: ctx.accounts.bid_wall.to_account_info(),  
126         },  
127         &[  
128             b"bid_wall",  
129             ctx.accounts.base_mint.key().as_ref(),  
130             ctx.accounts.bid_wall.creator.as_ref(),  
131             ctx.accounts.bid_wall.nonce.to_le_bytes().as_ref(),  
132             &[ctx.accounts.bid_wall.pda_bump],  
133         ]),  
134     ))?;
```

While the quote tokens are correctly transferred to the authority and fee recipient, the rent for the bid wall account plus for the ATA can be claimed by any user who calls `close_bid_wall` after expiration or when the bid wall is depleted.

Exploit scenario

1. Alice creates a bid wall, paying rent for the bid wall account and its quote token ATA.
2. The bid wall expires or becomes depleted.
3. Bob calls `close_bid_wall` with himself as the `payer`.
4. Bob receives rent that should have been returned to Alice.

Bob can automate this by monitoring all bid wall accounts via `getProgramAccounts`, tracking their `created_timestamp` and `duration_seconds` fields, and automatically submitting `close_bid_wall` transactions as soon as they become closeable.

Recommendation

Restrict the close instruction to only the authority, or allow anyone to close the bid wall while preserving the current constraints but return the rent to the authority instead of the caller.

Partial solution 1.1

The issue was partially fixed by returning the bid wall account rent to the authority. However, the `bid_wall_quote_token_account` token account is still closed to the `payer`, so its rent refund is returned to the caller. Additionally, the `payer` account is unnecessary and could be removed.

[Go back to Findings Summary](#)

L2: TWAP accumulator uses new_observation for the prior interval, enabling retroactive per-minute weighting under 60s update gating

Low severity issue

Impact:	Medium	Likelihood:	Low
Target:	programs/futarchy/src/state/futarchy_amm.rs	Type:	Logic error

Description

`Pool::update_twap()` updates the TWAP accumulator by adding `new_observation * dt` for the entire elapsed time `dt` since the last TWAP update. This means the updated observation is applied retroactively to the time interval that occurred before the observation update.

This contradicts both the `TwapOracle` documentation (“Running sum of `slots_per_last_update * last_observation`”) and standard TWAP semantics, which integrate the previous observation over the elapsed interval and only apply the updated observation prospectively.

The `TwapOracle` comment explicitly describes the accumulator as a running sum of time-deltas multiplied by the last observation:

Listing 13. Excerpt from `programs/futarchy/src/state/futarchy_amm.rs`

```
1 #[derive(Default, Clone, Copy, Debug, AnchorDeserialize, AnchorSerialize,
2   InitSpace)]
3 pub struct TwapOracle {
4     /// Running sum of slots_per_last_update * last_observation.
5     ///
6     /// ...
7 }
```

However, `update_twap()` computes `new_observation` and then adds

`new_observation * slot_difference` to the aggregator:

Listing 14. Excerpt from `programs/futarchy/src/state/futarchy_amm.rs`

```
1 pub fn update_twap(&mut self, current_timestamp: i64) -> Result<Option<u128>>
2 {
3     /// ...
4     let price = (self.quote_reserves as u128 * crate::PRICE_SCALE) /
5     self.base_reserves as u128;
6     let new_observation = price
7
8     let twap_start_timestamp = oracle.created_at_timestamp +
9     oracle.start_delay_seconds as i64;
10
11     let new_aggregator = if current_timestamp <= twap_start_timestamp {
12         oracle.aggregator
13     } else {
14         let effective_last_updated_timestamp =
15             oracle.last_updated_timestamp.max(twap_start_timestamp);
16
17         let slot_difference: u128 = (current_timestamp -
18             effective_last_updated_timestamp)
19             .try_into()
20             .unwrap();
21
22         let weighted_observation =
23             new_observation.saturating_mul(slot_difference);
24
25         oracle.aggregator.wrapping_add(weighted_observation)
26     };
27     /// ...
28     Ok(Some(new_observation))
29 }
```

Importantly, TWAP updates are only triggered by swaps and are rate-limited to once per 60 seconds. The program calls `update_twap()` immediately before executing swaps:

Listing 15. Excerpt from `programs/futarchy/src/state/futarchy_amm.rs`

```
1 pub fn update_twap(&mut self, current_timestamp: i64) -> Result<Option<u128>>
2 {
3     /// ...
4     if current_timestamp < oracle.last_updated_timestamp + 60 {
5         return Ok(None);
6     }
7
8     if self.base_reserves == 0 || self.quote_reserves == 0 {
9         return Ok(None);
10    }
11
12    /// ...
13
14    Ok(Some(new_observation))
15 }
```

Because proposal finalization uses `get_twap()` from both pass and fail markets to decide proposal outcome, this retroactive weighting can bias proposal outcomes at a per-minute granularity: within each 60-second window, price changes caused by swaps can be effectively applied backwards over the whole window at the time the next TWAP update is recorded.

`finalize_proposal` uses `get_twap()` to compute `pass_market_twap` and `fail_market_twap` and compares them against the passing threshold:

Listing 16. Excerpt from `finalize_proposal`

```
118 let calculate_twap = |amm: &Pool| -> Result<u128> {
119     let seconds_passed = amm.oracle.last_updated_timestamp -
120         proposal.timestamp_enqueued;
121     require_gte!(
122         seconds_passed,
123         proposal.duration_in_seconds as i64,
124         FutarchyError::MarketsTooYoung
125     );
126
127     amm.get_twap()
```

```
128 };
```

Exploit scenario

1. Alice triggers a swap at time T_0 that records a TWAP update (or waits for one to occur naturally).
2. Immediately after T_0 (still within the next 60 seconds), Alice performs swaps to move the pool price in her desired direction and keeps it there (e.g., by trading and/or providing offsetting trades).
 - Because `update_twap()` returns early for `current_timestamp < last_updated_timestamp + 60`, these swaps can change the live price without recording intermediate TWAP updates.
3. At time $T_1 \geq T_0 + 60$, Alice (or anyone) triggers the next swap. Before executing that swap, the program calls `update_twap()` and computes `new_observation` from the current pre-swap price at T_1 .
4. The program then updates the accumulator as `aggregator += new_observation * (T1 - T0)`, effectively applying the end-of-window observation retroactively across the entire previous 60-second window.
5. Repeating this pattern across windows can bias `get_twap()` in the pass/fail markets, which is used in `finalize_proposal` to decide proposal outcome.

Recommendation

Use the prior observation for the entire interval instead of the new observation.

Fix 1.1

The issue was fixed by using the prior observation for the entire interval instead of the new observation.

[Go back to Findings Summary](#)

W1: Compile-time gated admin check allows unauthorized fee collection when the production feature is disabled

Impact:	Warning	Likelihood:	N/A
Target:	programs/conditional_vault/src/instructions/add_metadata_to_conditional_tokens.rs, programs/futarchy/src/instructions/collect_fees.rs	Type:	Data validation

Description

The admin authorization is compiled away behind a feature flag instead of being enforced at runtime. In `CollectFees::validate()`, the key check is wrapped in `#[cfg(feature = "production")]`. When the binary is built without the production Cargo feature (the default per `programs/futarchy/Cargo.toml` has `default = []`), the check is omitted entirely, and any signer can pass validation.

This can be problematic for two reasons:

- By mistake, the program can be built without the production feature flag, and any signer can pass validation; and
- The protocol has less protection in case of private key compromise. The protocol will need to redeploy the whole program as the reference public key is compiled into the binary. A better approach is to make the admin key a parameter in a global configuration account.

Recommendation

Enforce admin authorization by storing the admin key in a global configuration account.

Acknowledgment 1.1

The issue is acknowledged by the client, with the following comment.

Finding is valid. We plan to invert the feature flag so admin checks are enabled by default and only disabled via an explicit development feature, eliminating the risk of accidentally shipping without them. Mainnet builds already pass features: "production" via CI, so deployed binaries are not affected today.

— MetaDAO team

[Go back to Findings Summary](#)

W2: Multiple instructions allow zero-amount operations causing unnecessary state updates and event emissions

Impact:	Warning	Likelihood:	N/A
Target:	programs/conditional_vault/src/instructions/{split_tokens, merge_tokens, redeem_tokens}.rs; programs/{futarchy, bid_wall}/src/instructions/collect_fees.rs	Type:	Logic error

Description

Several instructions across the Conditional Vault, Futarchy, and Bid Wall programs do not validate that the input amount is non-zero before processing. This allows callers to execute no-op transactions that still modify on-chain state (incrementing sequence numbers) and emit events, wasting compute units and polluting event logs.

Conditional Vault:

- `split_tokens`: Accepts `amount = 0`, performs zero-amount token transfer and mints, increments `vault.seq_num`, and emits `SplitTokensEvent`.

Listing 17. Excerpt from `split_tokens`

```
4 pub fn handle_split_tokens(ctx: Context<'_, '_, 'c, 'info, Self>, amount: u64)
  -> Result<> {
```

- `merge_tokens`: Accepts `amount = 0`, performs zero-amount burns and transfers, increments `vault.seq_num`, and emits `MergeTokensEvent`.

Listing 18. Excerpt from merge_tokens

```
4 pub fn handle_merge_tokens(ctx: Context<'_, '_, 'c, 'info, Self>, amount: u64)
  -> Result<()> {
```

- **redeem_tokens**: Can be called when user has zero conditional token balances, performs zero-amount burns and transfers, increments **vault.seq_num**, and emits **RedeemTokensEvent**.

Listing 19. Excerpt from redeem_tokens

```
13 pub fn handle_redeem_tokens(ctx: Context<'_, '_, 'c, 'info, Self>) ->
    Result<()> {
```

Futarchy:

- **collect_fees**: Does not check if **base_protocol_fee_balance** and **quote_protocol_fee_balance** are both zero. When called with zero fees, it resets both balances to 0 (redundant write), increments **dao.seq_num**, emits **CollectFeesEvent**, and performs two zero-amount token transfers.

Listing 20. Excerpt from collect_fees

```
52 let base_fee_balance = spot.base_protocol_fee_balance;
53 let quote_fee_balance = spot.quote_protocol_fee_balance;
54
55 spot.base_protocol_fee_balance = 0;
56 spot.quote_protocol_fee_balance = 0;
```

Bid Wall:

- **collect_fees**: Does not check if **fees_collected == 0**. When called with zero fees, it performs a zero-amount transfer, increments **bid_wall.seq_num**, emits **BidWallFeesCollectedEvent**, and resets **fees_collected** to 0 (redundant write).

Listing 21. Excerpt from collect_fees

```
38 pub fn handle(ctx: Context<Self>) -> Result<()> {
39     // transfer fees collected to fee recipient
40     token::transfer(
41         CpiContext::new_with_signer(
42             ctx.accounts.token_program.to_account_info(),
43             Transfer {
44                 from:
45                 ctx.accounts.bid_wall_quote_token_account.to_account_info(),
46                 to: ctx
47                     .accounts
48                     .fee_recipient_quote_token_account
49                     .to_account_info(),
50                 authority: ctx.accounts.bid_wall.to_account_info(),
51             },
52             &&[
53                 b"bid_wall",
54                 ctx.accounts.bid_wall.base_mint.as_ref(),
55                 ctx.accounts.bid_wall.creator.as_ref(),
56                 ctx.accounts.bid_wall.nonce.to_le_bytes().as_ref(),
57                 &[ctx.accounts.bid_wall.pda_bump],
58             ],
59             ctx.accounts.bid_wall.fees_collected,
60         )?;
61
62     // reload bid wall quote token account to get updated amount for event
63     ctx.accounts.bid_wall_quote_token_account.reload()?;
64     ctx.accounts.bid_wall.seq_num += 1;
65
66     emit_cpi!(BidWallFeesCollectedEvent {
67         common: CommonFields::new(&Clock::get()?,
68             ctx.accounts.bid_wall.seq_num),
69         bid_wall: ctx.accounts.bid_wall.key(),
70         fees_collected: ctx.accounts.bid_wall.fees_collected,
71         post_bid_wall_quote_token_account_amount: ctx
72             .accounts
73             .bid_wall_quote_token_account
74             .amount,
75     });
76     ctx.accounts.bid_wall.fees_collected = 0;
```

Recommendation

Prevent no-op sequence number incrementation by adding a check for 0 amount transfers, mint, burns.

Acknowledgment 1.1

The issue is acknowledged by the client, with the following comment.

Finding is valid. Our primary concern at the moment is as small of a codebase as possible. We will address this as part of a future update.

— MetaDAO team

[Go back to Findings Summary](#)

W3: Missing resolved check in `split_tokens` and `merge_tokens` allows operations on resolved questions

Impact:	Warning	Likelihood:	N/A
Target:	programs/conditional_vault/src/instructions/merge_tokens.rs, programs/conditional_vault/src/instructions/split_tokens.rs	Type:	Logic error

Description

The `split_tokens` and `merge_tokens` instructions allow users to split and merge conditional tokens for questions that have already been resolved. Once a question is resolved, users should only be able to redeem their conditional tokens based on the outcome, not continue to split or merge them. This operation is unnecessary and can cause confusion about the state of the prediction market.

The issue occurs in both `handle_split_tokens` and `handle_merge_tokens` functions, which do not check whether the question is resolved before processing the operations. The `Question` struct provides an `is_resolved()` method that checks if `payout_denominator != 0`, but this check is not performed in these instructions.

Recommendation

Allow the `merge_tokens` and `split_tokens` instructions to only be used for questions that are not resolved.

Fix 1.1

The issue was fixed by checking if the question is resolved before processing the operations.

[Go back to Findings Summary](#)

W4: DAO pass thresholds are not snapshotted per-proposal, allowing governance to change proposal finalization criteria mid-flight

Impact:	Warning	Likelihood:	N/A
Target:	programs/futarchy/src/instructions/finalize_proposal.rs, programs/futarchy/src/instructions/update_dao.rs	Type:	Logic error

Description

The DAO configuration can be updated via `update_dao`, including the key parameters that determine whether a proposal passes (`pass_threshold_bps` and `team_sponsored_pass_threshold_bps`).

However, `finalize_proposal` reads these thresholds directly from the current `dao` account at finalization time, rather than using values snapshotted when the proposal was created or launched:

Listing 22. Excerpt from `finalize_proposal`

```
142 let threshold_bps = if proposal.is_team_sponsored {
143     dao.team_sponsored_pass_threshold_bps
144 } else {
145     dao.pass_threshold_bps as i16
146 };
147
148 // this can't overflow because each twap can only be MAX_PRICE (~1e31),
149 // MAX_BPS + pass_threshold_bps is at most 1e5, and a u128 can hold
150 // 1e38
151
152 let threshold = fail_market_twap * u128::try_from(MAX_BPS as i16 +
    threshold_bps).unwrap()
153     / MAX_BPS as u128;
154
155 let (new_proposal_state, payout_numerators) = if pass_market_twap >
    threshold {
```

```
156     (ProposalState::Passed, vec![0, 1])
157 } else {
158     (ProposalState::Failed, vec![1, 0])
159 };
```

This creates a governance “footgun”: during an active proposal (i.e., while Futarchy markets are running), governance can change the passing rule right before finalization, potentially flipping the outcome even if market behavior was based on the prior threshold.

Recommendation

Snapshot `pass_threshold_bps` and `team_sponsored_pass_threshold_bps` into the `Proposal` and use the snapshotted values in `finalize_proposal`. Or, disallow updating these fields while DAO is in a Futarchy mode.

Fix 1.1

The issue was fixed by allowing the dao to be updated only in the Spot state.

[Go back to Findings Summary](#)

W5: Hardcoded fee recipient address prevents rotating the Bid Wall fee destination without redeploying

Impact:	Warning	Likelihood:	N/A
Target:	programs/v07_launchpad/src/lib.rs	Type:	Code quality

Description

The `fee_recipient` address is fixed instead of being stored in an on-chain configuration. Specifically, `fee_recipient::id()` is defined as a fixed `declare_id!()`.

This design choice can be problematic in case the fee recipient needs to be changed:

Listing 23. Excerpt from lib

```
55 pub mod fee_recipient {
56     use anchor_lang::prelude::declare_id;
57
58     // MetaDAO multisig vault
59     declare_id!("6awyHmshBGVjJ3ozdSJdyDE1CTAXUwrpNMaRGMSb4sf");
60 }
```

Recommendation

Store the fee recipient key in an on-chain configuration account and validate it at runtime in the `CompleteLaunch` instruction. Implement an authorized mechanism to update the stored key.

Acknowledgment 1.1

The client acknowledged the finding and provided the following comment:

We are keeping the codebase intentionally thin. Once we settle, we will add these kinds of configurations.

— MetaDAO team

[Go back to Findings Summary](#)

W6: Single-step authority transfer

Impact:	Warning	Likelihood:	N/A
Target:	programs/price_based_performance_package/src/instructions/change_performance_package_authority.rs	Type:	Code quality

Description

The `change_performance_package_authority` instruction updates `performance_package.performance_package_authority` in a single step. The instruction requires only the current authority to sign, and it does not require acceptance by the new authority:

Listing 24. Excerpt from `change_performance_package_authority`

```
12 #[derive(Accounts)]
13 pub struct ChangePerformancePackageAuthority<'info> {
14     #[account(mut)]
15     pub performance_package: Account<'info, PerformancePackage>,
16
17     #[account(address = performance_package.performance_package_authority @
18         PriceBasedPerformancePackageError::UnauthorizedLockerAuthority)]
19     pub current_authority: Signer<'info>,
20 }
```

The handler then writes the new authority directly:

Listing 25. Excerpt from `change_performance_package_authority`

```
31 let ChangePerformancePackageAuthorityParams {
32     new_performance_package_authority: new_locker_authority,
33 } = params;
34
35 let clock = Clock::get()?;
36 let old_authority = performance_package.performance_package_authority;
37
38 // Update the locker authority
```

```
39 performance_package.performance_package_authority = new_locker_authority;
```

If the current authority specifies an incorrect `new_performance_package_authority`, the performance package becomes controlled by that incorrect key, and the protocol cannot correct it without cooperation from the new key.

Recommendation

Implement a two-step authority transfer to safely update the performance package authority.

Acknowledgment 1.1

The client acknowledged the finding and provided the following comment:

Again, not absolutely necessary for our use-case, as the authority is the DAO's Squads Multisig Vault, meaning that we need to pass a futarchy proposal to change the authority.

— MetaDAO team

[Go back to Findings Summary](#)

I1: $O(N^2)$ linear search in `redeem_tokens` causes unnecessary compute usage

Impact:	Info	Likelihood:	N/A
Target:	programs/conditional_vault/src/instructions/redeem_tokens.rs	Type:	Code quality

Description

The `redeem_tokens` instruction computes each outcome's payout by iterating over the conditional mints and the corresponding user token accounts. For every pair, it derives a `payout_index` by linearly searching `vault.conditional_token_mints` using `.position()`. However, the account parser `get_mints_and_user_token_accounts` has already enforced both membership and order of conditional mints against `vault.conditional_token_mints`, making the current loop index directly map to the correct payout entry. Repeating an $O(N)$ search inside an $O(N)$ loop yields $O(N^2)$ behavior and unnecessarily high compute usage.

The issue occurs in the `handle_redeem_tokens` function, where a linear search is performed for each iteration:

Listing 26. Excerpt from `redeem_tokens`

```
49 // this is safe because we check that every conditional mint is a part of the
    vault
50 let payout_index = vault
51     .conditional_token_mints
52     .iter()
53     .position(|mint| mint == &conditional_mint.key())
54     .unwrap();
```

The order guarantee that makes the search unnecessary is enforced during account parsing in `get_mints_and_user_token_accounts`:

Listing 27. Excerpt from common

```
40 for i in 0..expected_num_conditional_tokens {
41   let conditional_token_mint = next_account_info(remaining_accs)?;
42   require_eq!(
43     ctx.accounts.vault.conditional_token_mints[i],
44     conditional_token_mint.key(),
45     VaultError::ConditionalMintMismatch
46   );
47
48   // really, this should never fail because we initialize mints when we
   initialize the vault
49   conditional_token_mints.push(
50     Account::<Mint>::try_from(conditional_token_mint)
51       .or(Err(VaultError::BadConditionalMint))?,
52   );
53 }
```

The complexity impact grows quadratically with the number of outcomes N for a given vault, as the linear search is performed for each outcome in the loop.

Recommendation

Replace the linear search with a direct index lookup using the loop index.

Fix 1.1

The issue was fixed by removing the unnecessary linear search within the loop.

[Go back to Findings Summary](#)

I2: Missing validation in `initialize_dao` allows identical base and quote mints causing accounting inconsistencies

Impact:	Info	Likelihood:	N/A
Target:	programs/futarchy/src/instructions/initialize_dao.rs	Type:	Code quality

Description

The `initialize_dao` instruction creates two AMM vaults as associated token accounts (ATAs) for the DAO PDA: one for `base_mint` and one for `quote_mint`. There is no runtime or account constraint enforcing that these two mints are distinct. Because ATA derivation is unique per (mint, authority, token program), setting `base_mint == quote_mint` collapses both `futarchy_amm_base_vault` and `futarchy_amm_quote_vault` to the same on-chain SPL Token account.

Downstream instructions (`provide_liquidity`, `spot_swap`, and `withdraw_liquidity`) require user and vault accounts by mint. With equal mints, callers can (and the program will) use the same account for both base and quote. This causes all "base" and "quote" transfers to operate on a single SPL account while the in-memory AMM state (`Pool { base_reserves, quote_reserves }`) updates two separate reserve numbers. This violates the program's intended invariants that reserves correspond to distinct assets, potentially enabling accounting inconsistencies or value leakage depending on operation ordering and state.

Recommendation

Add a runtime check in the `initialize_dao` instruction to ensure that `base_mint` and `quote_mint` are distinct. Reject the transaction if `base_mint.key() == quote_mint.key()` to prevent the creation of a DAO with

identical base and quote mints.

Fix 1.1

The issue was fixed by preventing the creation of a DAO with identical base and quote mints.

[Go back to Findings Summary](#)

I3: Incorrect error message in initialize_proposal contradicts Active status validation

Impact:	Info	Likelihood:	N/A
Target:	programs/futarchy/src/instructions/initialize_proposal.rs	Type:	Code quality

Description

The `initialize_proposal` instruction validates that the Squads proposal is in `Active` status before allowing proposal initialization. However, the error message for `InvalidSquadsProposalStatus` incorrectly states "Squads proposal must be in Draft status", which contradicts the actual validation logic that requires `Active` status.

This mismatch between the validation logic and the error message can cause confusion for developers and users who encounter this error.

Recommendation

Change the error message to "Squads proposal must be in Active status" to match the validation.

Fix 1.1

The issue was fixed by changing the error message to "Squads proposal must be in Active status" to match the validation.

[Go back to Findings Summary](#)

I4: Performance packages cannot be closed after all tranches are unlocked, preventing rent recovery

Impact:	Info	Likelihood:	N/A
Target:	programs/price_based_performance_package/src/instructions/complete_unlock.rs	Type:	Code quality

Description

The `price_based_performance_package` program initializes a `PerformancePackage` account and a dedicated token vault to custody locked tokens:

However, the program does not expose an instruction to close the `PerformancePackage` account after all tokens have been unlocked and transferred. As a result, once a performance package has fully vested (all tranches are unlocked), the account remains on-chain and its rent cannot be reclaimed.

Listing 28. Excerpt from complete_unlock

```
12 #[account(mut, has_one = token_mint, has_one =  
    performance_package_token_vault)]  
13 pub performance_package: Box<Account<'info, PerformancePackage>>,
```

Recommendation

Add an instruction to close a fully unlocked performance package and reclaim rent.

Acknowledgment 1.1

The client acknowledged the finding and provided the following comment:

We have a preference for as small a program as possible, and there are not a lot of PPs currently. Once comfortable, we will add close instructions.

— MetaDAO team

[Go back to Findings Summary](#)

I5: Recipient and authority can be set to the same key

Impact:	Info	Likelihood:	N/A
Target:	programs/price_based_performance_package/src/instructions/propose_change.rs, price_based_performance_package/src/instructions/execute_change.rs	Type:	Code quality

Description

The program implements a two-party change-request mechanism where one party proposes a change and the other party executes it. `ProposeChange` allows either the current recipient or the current authority to propose a change request:

Listing 29. Excerpt from propose_change

```
39 pub fn validate(&self) -> Result<> {
40     if self.proposer.key() != self.performance_package.recipient
41         && self.proposer.key() !=
42         self.performance_package.performance_package_authority
43     {
44         msg!("proposer ({{}}) is not the token recipient ({{}}) or performance
45             package authority ({{}})", self.proposer.key(),
46             self.performance_package.recipient,
47             self.performance_package.performance_package_authority);
48         return
49         Err(PriceBasedPerformancePackageError::UnauthorizedChangeRequest.into());
50     }
```

`ExecuteChange` then requires the opposite party (recipient vs. authority) to execute the change:

Listing 30. Excerpt from `execute_change`

```
25 pub fn validate(&self) -> Result<()> {
26     if self.change_request.proposer_type == ProposerType::Recipient {
27         // If recipient proposed, locker authority must execute
28         require_keys_eq!(
29             self.executor.key(),
30             self.performance_package.performance_package_authority,
31             PriceBasedPerformancePackageError::UnauthorizedLockerAuthority
32         );
33     } else if self.change_request.proposer_type == ProposerType::Authority {
34         // If authority proposed, recipient must execute
35         require_keys_eq!(
36             self.executor.key(),
37             self.performance_package.recipient,
38             PriceBasedPerformancePackageError::UnauthorizedChangeRequest
39         );

```

However, `InitializePerformancePackage` does not enforce that `grantee` (stored as `recipient`) and `performance_package_authority` are distinct keys.

If `recipient == performance_package_authority`, the same signer can both propose and execute change requests, which defeats the purpose of the two-party confirmation mechanism.

Recommendation

Enforce `recipient != performance_package_authority` during performance package initialization and when updating either field via change requests.

Fix 1.1

The issue was fixed by adding a check to ensure that `recipient` and `performance_package_authority` are distinct keys during initialization, authority change, and change request execution.

[Go back to Findings Summary](#)

I6: Anyone can complete unlock, allowing third parties to choose the TWAP evaluation time

Impact:	Info	Likelihood:	N/A
Target:	programs/price_based_performance_package/src/instructions/complete_unlock.rs	Type:	Logic error

Description

The `complete_unlock` instruction does not require the performance package recipient or the performance package authority to sign. Instead, the only required signer is an arbitrary `payer` (used to fund `init_if_needed` for the recipient's Associated Token Account (ATA)):

Listing 31. Excerpt from `complete_unlock`

```
9  #[derive(Accounts)]
10 #[event_cpi]
11 pub struct CompleteUnlock<'info> {
12     #[account(mut, has_one = token_mint, has_one =
13         performance_package_token_vault)]
14     pub performance_package: Box<Account<'info, PerformancePackage>>,
15     /// CHECK: We will read the aggregator value from this account
16     #[account(address = performance_package.oracle_config.oracle_account)]
17     pub oracle_account: UncheckedAccount<'info>,
18     /// The token account where locked tokens are stored
19     #[account(mut)]
20     pub performance_package_token_vault: Box<Account<'info, TokenAccount>>,
21     /// The token mint - validated via has_one constraint on locker
22     pub token_mint: Account<'info, Mint>,
23     /// The recipient's ATA where tokens will be sent - created if needed
24     #[account(
25         init_if_needed,
26         payer = payer,
27         associated_token::mint = token_mint,
```

```

31         associated_token::authority = token_recipient
32     )]
33     pub recipient_token_account: Box<Account<'info, TokenAccount>>,
34
35     /// CHECK: validated to match locker.token_recipient
36     #[account(address = performance_package.recipient @
37         PriceBasedPerformancePackageError::UnauthorizedChangeRequest)]
38     pub token_recipient: UncheckedAccount<'info>,
39
40     /// Payer for creating the ATA if needed
41     #[account(mut)]
42     pub payer: Signer<'info>,
43
44     pub system_program: Program<'info, System>,
45
46     pub token_program: Program<'info, Token>,
47     pub associated_token_program: Program<'info, AssociatedToken>,
48 }

```

Once a performance package is in the **Unlocking** state, any third party can call **complete_unlock** and finalize the unlock at a time of their choosing (subject to the TWAP period constraints). This can affect how many tranches unlock in that cycle and can force the recipient to restart the unlock process.

Exploit scenario

Alice, a third party, wants to influence the time at which a performance package unlock cycle is finalized. Bob, the token recipient, has started an unlock cycle by calling **start_unlock**.

1. Bob calls **start_unlock**, placing the performance package into the **Unlocking** state;
2. Once the TWAP period elapses, Alice monitors oracle updates and calls **complete_unlock** at a time that produces a less favorable TWAP (e.g., unlocking fewer tranches than Bob intended); and
3. The instruction resets the performance package state back to **Locked**, requiring Bob to call **start_unlock** again to begin a new unlock cycle.

Recommendation

Require the token recipient or the performance package authority to authorize `complete_unlock`. Alternatively, restrict `complete_unlock` to the token recipient.

Acknowledgment 1.1

The client stated that the current behavior is intended and will not be changed and provided the following comment:

Permissionless complete_unlock is by design. The TWAP smoothing period (minimum 1 day) limits how much timing manipulation matters in any single cycle, and already-unlocked tranches cannot be reversed. In the worst case, a third party forces a slightly less favorable evaluation in certain scenarios. However, the evaluation time is intended to be as close to exact as possible, and allowing anyone to crank it gives us that ability.

Restricting complete_unlock to the recipient would introduce a liveness risk: if the recipient becomes unresponsive, tokens remain locked indefinitely with no way to complete the cycle. Keeping it permissionless ensures the unlock process can always make forward progress.

— MetaDAO team

[Go back to Findings Summary](#)

Report Revision 1.1

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Matej Hyčko	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

Overview

Since there were no comprehensive changes in this revision, the complete overview is listed in the Executive Summary section [Revision 1.1](#).

Findings

The following section presents the list of findings discovered in this revision. For the complete list of all findings, [Go back to Findings Summary](#)

W7: Proposal finalization depends on a post-deadline TWAP update

Impact:	Warning	Likelihood:	N/A
Target:	programs/futarchy/src/instructions/finalize_proposal.rs	Type:	Logic error

Description

The `finalize_proposal` instruction correctly checks that the time elapsed since proposal launch is at least the configured proposal duration. However, it also checks that the oracle `last_updated_timestamp` is at least one full proposal duration after `proposal.timestamp_enqueued`.

Listing 32. Excerpt from

programs/futarchy/src/instructions/finalize_proposal.rs

```
1 let calculate_twap = |amm: &Pool| -> Result<u128> {
2     let seconds_passed = amm.oracle.last_updated_timestamp -
        proposal.timestamp_enqueued;
3
4     require_gte!(
5         seconds_passed,
6         proposal.duration_in_seconds as i64,
7         FutarchyError::MarketsTooYoung
8     );
9
10    amm.get_twap(clock.unix_timestamp)
11 };
```

Because `last_updated_timestamp` only advances on oracle updates (typically from trades), a proposal may be impossible to finalize unless at least one update occurs at or after the proposal deadline. This makes proposal finalization depend on post-deadline market activity, which is stricter than the intended “time since launch” duration check.

Recommendation

- Add a permissionless "heartbeat" instruction that updates the oracle/TWAP without requiring a trade; or
- Remove or relax the `last_updated_timestamp - timestamp_enqueued >= duration` check and instead require only that the oracle has been updated at least once since launch (e.g., `last_updated_timestamp != timestamp_enqueued`), while keeping the elapsed-time duration check. Additionally, the `get_twap()` method considers the remaining time from `last_updated_timestamp`, which means that if there are no trades for some period before proposal finalization, the method still accounts for the latest observation in the TWAP calculation.

Fix 1.2

The issue was fixed by relaxing the check, which now verifies that at least one oracle update happened after the TWAP update delay (`start_delay_seconds`) ended.

[Go back to Findings Summary](#)

Report Revision 1.2

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Matej Hyčko	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

Overview

Since there were no comprehensive changes in this revision, the complete overview is listed in the Executive Summary section [Revision 1.2](#).

Report Revision 2.0

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Matej Hyčko	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

MetaDAO The fundraising and governance platform is a futarchy-based governance platform on Solana that enables decision-making through prediction markets.

Mint Governor manages delegated minting authority for SPL tokens. It places mint authority under program control so that an admin can authorize specific accounts to mint, update or remove those mint permissions over time, and reclaim authority when needed. This allows token issuance to be governed through explicit onchain authorization records rather than leaving unrestricted mint control in a single externally owned account.

Performance Package V2 is a token compensation and vesting program that escrows tokens for a recipient and releases them according to predefined package conditions. It is designed to support structured token grants with program-enforced unlock logic, so that teams can distribute compensation transparently and recipients can rely on onchain enforcement rather than offchain promises alone.

Together, these programs extend the governance ecosystem by controlling token issuance and managing long-term incentive alignment. Mint Governor governs who may expand token supply, while Performance Package V2

governs how allocated tokens become claimable over time.

Trust Model

Mint Governor

The Mint Governor program manages delegated minting authority for SPL tokens.

In the Mint Governor program, users must trust that:

- the **Mint Governor Admin** will only approve legitimate mint authorities and will not authorize abusive or excessive token issuance;
- the **Mint Governor Admin** will update or revoke mint permissions fairly and in accordance with the intended token issuance policy;
- authorized **Mint Authorities** will only mint tokens for legitimate purposes and within the scope of their expected permissions;
- the **Mint Governor Admin** will not reclaim, redirect, or misuse mint authority in a way that undermines tokenholder expectations; and
- the governed **Mint** is configured correctly, since control over that mint determines the token supply.

Performance Package V2

The Performance Package V2 program manages token compensation packages with program-enforced release conditions.

In the Performance Package V2 program, users must trust that:

- the **Package Authority** and **Recipient** will act honestly in any workflow that requires mutual consent for changes;

- any configured milestone, performance, or oracle-based unlock conditions are accurate and not manipulable;
- the **Package Authority** will not use any permitted update, or cancellation flow to unfairly deprive the recipient of earned tokens; and
- the package is funded with the correct **Mint** and token amount, since those parameters determine the recipient's real entitlement.

Findings

The following section presents the list of findings discovered in this revision. For the complete list of all findings, [Go back to Findings Summary](#)

W8: Rent destination is not strictly enforced to be the initial rent payer

Impact:	Warning	Likelihood:	N/A
Target:	programs/mint_governor/src/instructions/remove_mint_authority.rs	Type:	Logic error

Description

The `remove_mint_authority` instruction allows the `MintGovernor` admin to remove `MintAuthority`. The account is closed by removing its data and returning rent. However, the rent is not strictly returned to the initial rent payer, but to `rent_destination`, which is an unconstrained account. This can result in a situation where the rent is paid by the user but collected by the admin, causing the user to lose a small amount of funds.

Recommendation

Restrict the `rent_destination` to be the initial rent payer.

Acknowledgment 2.1

The client acknowledged the finding with the following comment.

I intentionally didn't want to add this extra piece of complexity to the codebase. Considering these transfers usually happen as a result of a market decision, I consider this single-step process safe, as there's an entire decision mechanism behind it.

— MetaDAO team

[Go back to Findings Summary](#)

W9: Single step authority transfer process is considered unsafe

Impact:	Warning	Likelihood:	N/A
Target:	programs/mint_governor/src/instructions/update_mint_governor_admin.rs, programs/performance_package_v2/src/instructions/change_authority.rs	Type:	Logic error

Description

The `update_mint_governor_admin` instruction in the MintGovernor program allows the current admin to appoint a new admin.

The `change_authority` instruction in the PerformancePackageV2 program allows the current authority to appoint a new authority for a performance package.

In both cases, the implemented process is single-step, once the new authority is appointed, there is no way to revert the change.

Recommendation

Implement a two-step transfer process in which the new admin accepts the role, while the current admin can still revert the change.

Acknowledgment 2.1

The client acknowledged the finding with the following comment.

We don't track who the original creator is, and we accept that a small amount of rent can be lost this way. Ultimately,

the admin will decide who the rent goes to.

— MetaDAO team

[Go back to Findings Summary](#)

I7: Safety checks are disabled when the production feature is not enabled

Impact:	Info	Likelihood:	N/A
Target:	programs/performance_package_v2/src/instructions/close_performance_package.rs	Type:	Data validation

Description

Some safety checks are compiled away behind the `production` feature flag instead of being enforced at runtime. For example, in `ClosePerformancePackage::validate()`, the admin authorization check is wrapped in `#[cfg(feature = "production")]`. When the program is built without the `production` feature enabled, this check is omitted and the `admin` account is not validated against the expected privileged key.

Recommendation

Enforce safety checks at runtime instead of compiling them out behind the `production` feature flag. If different behavior is required for development, gate the relaxed path behind an explicit development-only configuration.

Acknowledgment 2.1

The client acknowledged the finding with the following comment.

Currently we do this accross multiple programs for simpler testing. In the future we will have all of these admin checks on by default, with a special "development" flag for local testing. We simply didn't get around to that.

— MetaDAO team

[Go back to Findings Summary](#)

Report Revision 2.1

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Matej Hyčko	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

Overview

Since there were no comprehensive changes in this revision, the complete overview is listed in the Executive Summary section [Revision 2.1](#).

Appendix A: How to cite

Please cite this document as:

[Ackee Blockchain Security](#), Audit Report | MetaDAO: The fundraising and governance platform, April 1, 2026.

Appendix B: Trident Findings

This section details the results and methodology of fuzz testing performed using the [Trident](#) framework during the audit.

B.1. Fuzzing

Mainnet State Forking: External program accounts (Squads multisig program, program configs) are fetched from mainnet and cached locally using Trident's fork feature. This ensures realistic integration testing against production-deployed dependencies without mocking.

Time Manipulation: The fuzzer uses `forward_in_time()` to simulate passage of time between trades, enabling TWAP oracle updates and proposal duration requirements. For finalization testing, time is advanced past the proposal duration threshold.

TWAP Cranking: Before finalizing proposals, a minimal swap is executed to update the TWAP oracle observation, ensuring the finalization logic has fresh price data to evaluate pass/fail thresholds.

Balance Conservation Tracking: A snapshot of total underlying token supply is captured at initialization. Global invariants verify this total remains constant across all tracked owners and vault accounts throughout the fuzz run.

Conditional Token Auto-Split: When traders lack conditional token balances for swaps, the fuzzer automatically splits underlying tokens into pass/fail conditional tokens, enabling comprehensive coverage of conditional market trading paths.

Multi-Pool State Alignment: In Futarchy mode, invariants verify that aggregated pool states (spot + pass, spot + fail) align with actual DAO vault balances, accounting for the split/merge mechanics between underlying and

conditional tokens.

The following table lists all implemented execution flows in the [Trident](#) fuzz testing framework.

ID	Flow	Added
F1	fuzz_0: Alice provides liquidity with randomized amounts	1.0
F2	fuzz_0: Bob provides liquidity with randomized amounts	1.0
F3	fuzz_0: Alice withdraws liquidity with randomized amounts	1.0
F4	fuzz_0: Bob withdraws liquidity with randomized amounts	1.0
F5	fuzz_1: Alice performs spot swap with randomized buy/sell and size	1.0
F6	fuzz_1: Bob performs spot swap with randomized buy/sell and size	1.0
F7	fuzz_2: Alice performs conditional swap (Pass/Fail x Buy/Sell), with auto split when needed	1.0
F8	fuzz_2: Bob performs conditional swap (Pass/Fail x Buy/Sell), with auto split when needed	1.0
F9	fuzz_3: Alice performs conditional swap in Futarchy mode with auto split and time forwarding	1.0
F10	fuzz_3: Bob performs conditional swap in Futarchy mode with auto split and time forwarding	1.0
F11	fuzz_3: Alice performs spot swap in Futarchy mode with randomized buy/sell and time forwarding	1.0
F12	fuzz_3: Bob performs spot swap in Futarchy mode with randomized buy/sell and time forwarding	1.0

ID	Flow	Added
F13	fuzz_3: Finalize proposal after trading (time forward, TWAP crank, transition to Passed/Failed)	1.0
F14	fuzz_launchpad: StartLaunch or Fund (randomized amount, occasional wrong PDAs/ATAs) based on launch state	1.0
F15	fuzz_launchpad: CloseLaunch (often after time-forwarding past the launch period), then SetFundingRecordApproval for an existing FundingRecord	1.0
F16	fuzz_launchpad: SetFundingRecordApproval with randomized approved amount	1.0
F17	fuzz_launchpad: CompleteLaunch with typically correct authority and required vault/mint/metadata accounts	1.0
F18	fuzz_launchpad: Refund using a funder FundingRecord (occasional wrong funder quote ATA)	1.0
F19	fuzz_launchpad: Claim base tokens using a funder FundingRecord (occasional wrong funder base ATA)	1.0
F20	fuzz_launchpad: Post-complete extras: ClaimAdditionalTokenAllocation or InitializePerformancePackage when applicable	1.0
F21	fuzz_pbpp: StartUnlock for a recipient, typically with oracle + time set up to satisfy minUnlockTimestamp	1.0
F22	fuzz_pbpp: CompleteUnlock for a recipient, typically after time-forwarding past twapLengthSeconds and updating oracle data for tranche thresholds	1.0

ID	Flow	Added
F23	fuzz_pbpp: Admin bundle: ChangePerformancePackageAuthority or ProposeChange + ExecuteChange, with randomized proposer/executor	1.0

Table 4. Trident fuzzing flows

The following table lists all implemented invariant checks in the [Trident](#) fuzz testing framework.

ID	Invariant	Added	Status
IV1	fuzz_0: Provide liquidity cannot increase provider token balances	1.0	Success
IV2	fuzz_0: Provide liquidity increases pool reserves, total liquidity, and provider position liquidity	1.0	Success
IV3	fuzz_0: Withdraw liquidity moves balances/reserves in opposite expected direction	1.0	Success
IV4	fuzz_0: Cumulative user withdrawals never exceed cumulative deposits (base and quote)	1.0	Fail (C1)
IV5	fuzz_0: Spot reserves must equal DAO vault token balances	1.0	Success
IV6	fuzz_0: If total liquidity is zero, both reserves must be zero	1.0	Success
IV7	fuzz_1: Spot buy directionality holds (trader base up/quote down, pool base down/quote up)	1.0	Success

ID	Invariant	Added	Status
IV8	fuzz_1: Spot sell directionality holds (trader base down/quote up, pool base up/quote down)	1.0	Success
IV9	fuzz_1: DAO vault balances equal reserves plus accumulated protocol fees	1.0	Success
IV10	fuzz_1: If total liquidity is zero, both reserves must be zero	1.0	Success
IV11	fuzz_2: Conditional swap cannot increase trader input balance or decrease trader output balance	1.0	Success
IV12	fuzz_2: Conditional buy/sell reserve directionality holds for the selected market pool	1.0	Success
IV13	fuzz_2: Total base underlying supply across tracked owners plus vault ATA is conserved	1.0	Success
IV14	fuzz_2: Total quote underlying supply across tracked owners plus vault ATA is conserved	1.0	Success
IV15	fuzz_2: Pass and fail base conditional totals remain equal across tracked owners	1.0	Success
IV16	fuzz_2: Pass and fail quote conditional totals remain equal across tracked owners	1.0	Success
IV17	fuzz_3: Conditional swap cannot increase trader input balance or decrease trader output balance	1.0	Success
IV18	fuzz_3: Conditional buy/sell reserve directionality holds for the selected market pool	1.0	Success

ID	Invariant	Added	Status
IV19	fuzz_3: Spot buy directionality holds (trader base up/quote down)	1.0	Success
IV20	fuzz_3: Spot sell directionality holds (trader base down/quote up)	1.0	Success
IV21	fuzz_3: Total base underlying supply across tracked owners plus vault ATA is conserved	1.0	Success
IV22	fuzz_3: Total quote underlying supply across tracked owners plus vault ATA is conserved	1.0	Success
IV23	fuzz_3: Pass and fail base conditional totals remain equal across tracked owners	1.0	Success
IV24	fuzz_3: Pass and fail quote conditional totals remain equal across tracked owners	1.0	Success
IV25	fuzz_3: Futarchy vault alignment - spot+pass base leg state matches DAO balances	1.0	Success
IV26	fuzz_3: Futarchy vault alignment - spot+fail base leg state matches DAO balances	1.0	Success
IV27	fuzz_3: Futarchy vault alignment - spot+pass quote leg state matches DAO balances	1.0	Success
IV28	fuzz_3: Futarchy vault alignment - spot+fail quote leg state matches DAO balances	1.0	Success
IV29	fuzz_3: Finalize proposal must succeed and transition proposal to Passed or Failed	1.0	Success
IV30	fuzz_3: After finalize, DAO AMM must transition back to Spot state	1.0	Success
IV31	fuzz_3: Post-finalize spot base vault must align with spot reserves plus fees	1.0	Success

ID	Invariant	Added	Status
IV32	fuzz_3: Post-finalize spot quote vault must align with spot reserves plus fees	1.0	Success
IV33	fuzz_launchpad: InitializeLaunch derives launch and launch_signer PDAs from expected seeds, and vaults are the expected ATAs for launch_signer	1.0	Success
IV34	fuzz_launchpad: InitializeLaunch persists Launch configuration to match the provided args/accounts, and the initial state is Initialized	1.0	Success
IV35	fuzz_launchpad: InitializeLaunch leaves the quote vault unchanged, and increases the base vault by the expected minted total	1.0	Success
IV36	fuzz_launchpad: StartLaunch requires the configured Launch.launchAuthority	1.0	Success
IV37	fuzz_launchpad: StartLaunch transitions state from Initialized to Live	1.0	Success
IV38	fuzz_launchpad: StartLaunch sets unixTimestampStarted within the transaction time window	1.0	Success
IV39	fuzz_launchpad: SetFundingRecordApproval requires the configured Launch.launchAuthority	1.0	Success
IV40	fuzz_launchpad: SetFundingRecordApproval requires Closed state and a 2-day approval window	1.0	Success

ID	Invariant	Added	Status
IV41	fuzz_launchpad: SetFundingRecordApproval sets FundingRecord.approvedAmount ($\leftarrow$ committedAmount)	1.0	Success
IV42	fuzz_launchpad: SetFundingRecordApproval adjusts Launch.totalApprovedAmount by the approvedAmount delta	1.0	Success
IV43	fuzz_launchpad: Refund is valid only in Refunding or Complete state, and only once per FundingRecord	1.0	Success
IV44	fuzz_launchpad: Refund preserves Launch state and sets FundingRecord.isUsdcRefunded	1.0	Success
IV45	fuzz_launchpad: Refund amount is committedAmount, or committedAmount minus approvedAmount	1.0	Success
IV46	fuzz_launchpad: Refund token deltas match the refunded amount (vault down, funder up)	1.0	Success
IV47	fuzz_launchpad: Fund requires Live state, amount > 0, and a non-expired launch period	1.0	Success
IV48	fuzz_launchpad: Fund requires the configured launch_signer and launch_quote_vault accounts	1.0	Success
IV49	fuzz_launchpad: Fund creates or updates FundingRecord.committedAmount by amount	1.0	Success
IV50	fuzz_launchpad: Fund transfers amount from funder quote ATA to the launch quote vault	1.0	Success

ID	Invariant	Added	Status
IV51	fuzz_launchpad: Fund increases Launch.totalCommittedAmount by amount and keeps state Live	1.0	Success
IV52	fuzz_launchpad: CloseLaunch requires Live state and a completed launch period	1.0	Success
IV53	fuzz_launchpad: CloseLaunch transitions to Closed or Refunding based on minimumRaiseAmount	1.0	Success
IV54	fuzz_launchpad: CloseLaunch sets unixTimestampClosed within the transaction time window	1.0	Success
IV55	fuzz_launchpad: CompleteLaunch requires Closed state and a valid authority during the 2-day window	1.0	Success
IV56	fuzz_launchpad: CompleteLaunch transitions to Refunding when totalApprovedAmount < minimumRaiseAmount	1.0	Success
IV57	fuzz_launchpad: CompleteLaunch early-refund path leaves launch vault balances unchanged	1.0	Success
IV58	fuzz_launchpad: CompleteLaunch success path sets dao/daoVault and unixTimestampCompleted, and transitions to Complete	1.0	Success
IV59	fuzz_launchpad: CompleteLaunch keeps Launch totals unchanged (only distribution/state fields update)	1.0	Success

ID	Invariant	Added	Status
IV60	fuzz_launchpad: CompleteLaunch credits DAO treasury by the expected USDC allocation	1.0	Success
IV61	fuzz_launchpad: CompleteLaunch credits the bid wall by the expected USDC allocation, when funded	1.0	Success
IV62	fuzz_launchpad: CompleteLaunch leaves enough USDC for refunds, and enough base tokens for claims	1.0	Success
IV63	fuzz_launchpad: Claim requires Complete state and a not-yet-claimed FundingRecord	1.0	Success
IV64	fuzz_launchpad: Claim amount is proportional to approvedAmount over totalApprovedAmount	1.0	Success
IV65	fuzz_launchpad: Claim transfers the expected token amount from launch base vault to funder ATA	1.0	Success
IV66	fuzz_launchpad: Claim sets FundingRecord.isTokensClaimed and preserves Launch state/totals	1.0	Success
IV67	fuzz_launchpad: ClaimAdditionalTokenAllocation requires Complete state and unclaimed allocation	1.0	Success
IV68	fuzz_launchpad: ClaimAdditionalTokenAllocation requires the configured additionalTokensRecipient	1.0	Success

ID	Invariant	Added	Status
IV69	fuzz_launchpad: ClaimAdditionalTokenAllocation transfers additionalTokensAmount from vault to recipient	1.0	Success
IV70	fuzz_launchpad: ClaimAdditionalTokenAllocation flips additionalTokensClaimed and keeps state Complete	1.0	Success
IV71	fuzz_launchpad: InitializePerformancePackage requires Complete state and a missing performance package	1.0	Success
IV72	fuzz_launchpad: InitializePerformancePackage flips isPerformancePackageInitialized and keeps dao inputs consistent	1.0	Success
IV73	fuzz_launchpad: InitializePerformancePackage defines 5 tranches with thresholds 2x..32x launch price	1.0	Success
IV74	fuzz_launchpad: InitializePerformancePackage sets minUnlockTimestamp from unixTimestampCompleted and monthsUntillInsidersCanUnlock	1.0	Success
IV75	fuzz_launchpad: InitializePerformancePackage transfers performance package tokens from launch base vault to performance package vault	1.0	Success

ID	Invariant	Added	Status
IV76	fuzz_pbpp: InitializePerformancePackage persists inputs and starts Locked	1.0	Success
IV77	fuzz_pbpp: InitializePerformancePackage stores tranches 1:1, all locked, and totalTokenAmount matches sum	1.0	Success
IV78	fuzz_pbpp: InitializePerformancePackage requires minUnlockTimestamp to be in the future	1.0	Success
IV79	fuzz_pbpp: ChangePerformancePackageAuthority requires the current authority and updates authority to the input	1.0	Success
IV80	fuzz_pbpp: ProposeChange stores proposerType (recipient vs authority) and sets proposedAt within the transaction time window	1.0	Success
IV81	fuzz_pbpp: ProposeChange must not mutate the PerformancePackage state	1.0	Success
IV82	fuzz_pbpp: ExecuteChange requires the opposite party of proposerType and applies the requested change	1.0	Success
IV83	fuzz_pbpp: ExecuteChange increments seqNum and closes the ChangeRequest account	1.0	Success
IV84	fuzz_pbpp: StartUnlock requires the configured recipient and transitions from Locked to Unlocking	1.0	Success

ID	Invariant	Added	Status
IV85	fuzz_pbpp: StartUnlock snapshots oracle bytes into startAggregator/startTimestamp and does not move tokens	1.0	Success
IV86	fuzz_pbpp: StartUnlock requires unlock time bounds	1.0	Success
IV87	fuzz_pbpp: CompleteUnlock requires Unlocking state and TWAP time $\geq$ twapLengthSeconds	1.0	Success
IV88	fuzz_pbpp: CompleteUnlock updates tranche unlock flags and alreadyUnlockedAmount based on TWAP price	1.0	Success
IV89	fuzz_pbpp: CompleteUnlock transfers unlocked tokens from vault to recipient and returns to Locked	1.0	Success

Table 5. Trident fuzzing invariants

Appendix C: Trident Arena Findings

This section lists vulnerabilities identified by [Trident Arena](#), an automated vulnerability analysis system designed for Solana program reviews. Trident Arena uses multi-agent AI as one of its core analysis engines and layers additional checks on top, complementing manual review.

C.1. Discovered Findings

The following table contains true-positive findings identified by [Trident Arena](#). These findings are included regardless of whether they were also discovered independently by auditors during manual review.

Finding title	Severity	Reported	Discoverer
C1 : u128→u64 downcast during withdraw_liquidity corrupts share math and can cause division-by-zero	Critical	1.0	Trident Arena, Auditor
H1 : LP Position Seeds/Authority Mismatch Bricks Withdrawals	High	1.0	Trident Arena, Auditor
H2 : Oracle configuration can be changed during an unlock period, enabling TWAP manipulation	High	1.0	Trident Arena, Auditor
L2 : TWAP accumulator uses new_observation for the prior interval, enabling retroactive per-minute weighting under 60s update gating	Low	1.0	Trident Arena
W2 : Multiple instructions allow zero-amount operations causing unnecessary state updates and event emissions	Warning	1.0	Trident Arena, Auditor

Finding title	Severity	Reported	Discoverer
W3 : Missing resolved check in split_tokens and merge_tokens allows operations on resolved questions	Warning	1.0	Trident Arena, Auditor
W4 : DAO pass thresholds are not snapshotted per-proposal, allowing governance to change proposal finalization criteria mid-flight	Warning	1.0	Trident Arena, Auditor

Table 6. Table of findings identified by [Trident Arena](#)



Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz